

COMP 3069

Computer Graphics

Lecture 8: Cameras

Autumn 2024



The University of
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

0

Contents

- ◆ Need for a digital camera in rendering
- ◆ Essential components of a camera
- ◆ Model-Viewer Camera
- ◆ Fly-Through Camera



The University of
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

1

1

1

Examples of Camera Types

- ◆ A camera attached to a plane
- ◆ A camera attached to a helmet
- ◆ A camera that attaches to a moving crane and used for filming
- ◆ A camera that records a football match from above the pitch at the top of the stadium

在球场上方、看台顶部固定的摄像机，用来拍足球比赛（类似转播镜头）。

Cameras in This Module

一个模型查看相机（Model-Viewer Camera）和一个飞行相机（Fly-Through Camera）

- ◆ A Model-Viewer Camera and a Fly-Through Camera
- ◆ They are intuitively placed in the scene to render the geometry
- ◆ They can be used to interact with graphics and can be moved interactively
- ◆ They are used to view very specific points in the scene from specific angles

Essentials of a Camera

- ◆ A Position in 3D space 相机在3D空间的位置
- ◆ A Forward Direction 前方向
- ◆ A Right Direction 右方向
- ◆ An Up Direction 上方向
- ◆ A camera position is its location in world space. 相机的位置就是它在世界坐标系中的坐标
- ◆ Calculating a camera direction can be achieved using Euler angles, Pitch, Yaw and Roll 计算相机的方向可以使用欧拉角（Pitch, Yaw, Roll）完成

解释：

欧拉角定义三个方向旋转：

- Yaw (偏航)：左转/右转（绕 Y 轴）
- Pitch (俯仰)：抬头/低头（绕 X 轴）
- Roll (翻滚)：歪头（绕 Z 轴），一般不使用

4

4

在课程中只用两个：

- Yaw + Pitch 来计算 Front 向量
- Roll 通常保持为 0（不让相机倾斜）

数学意义：

你每次更新 yaw/pitch 时，就是重新计算 Front，然后根据 Front 计算 Right 和 Up。

1. Yaw (偏航) = 水平旋转 = α (图中的 α)

图中 α 是沿着水平圆环旋转的角度：

- 像你转头“左看 / 右看”
- 围绕 世界坐标系 Y 轴 旋转
- 控制相机左右方向
- 在你的摄像机代码里：

$$\text{Yaw} = \text{Yaw} + \text{offset} * \text{MouseSensitivity}$$

数学意义：

Yaw 控制 Front 向量的 x 和 z 分量：

$$\text{front}_x = \cos(\text{yaw}) \cos(\text{pitch})$$

$$\text{front}_z = \sin(\text{yaw}) \cos(\text{pitch})$$

图上对应：绕竖直线旋转的角度 α

3. Roll (翻滚) = γ 或图中的其他水平角 (通常不用)

图中 γ 是水平面的一个角度，但在相机中：

Roll = 相机“歪头”

- 围绕 Z 轴 旋转 (像把头倾斜)
- 游戏/应用几乎从不使用 (一般保持 0°)

因此我们通常只用：

- ✓ Yaw
- ✓ Pitch
- ✗ 不用 Roll

2. Pitch (俯仰) = 垂直旋转 = β (图中的 β)

图中 β 是向上抬头的角度 (沿垂直方向旋转)：

- 相机“抬头 / 低头”
- 围绕 相机的 X 轴 旋转
- 改变相机上下方向
- 在你的摄像机代码里：

$$\text{Pitch} = \text{Pitch} + \text{offset} * \text{MouseSensitivity}$$

数学意义：

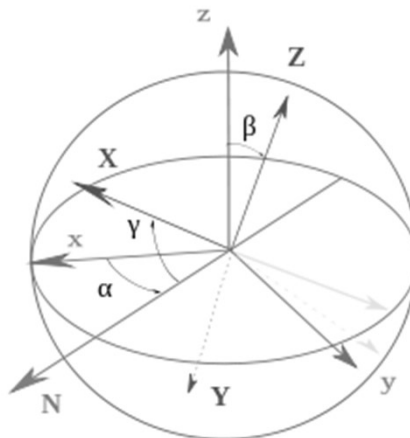
Pitch 控制 Front 的 y 分量：

$$\text{front}_y = \sin(\text{pitch})$$

图上对应：球面上向上旋转的 β 角

Pitch 必须限制在 $[-89^\circ, 89^\circ]$ ，否则会翻转。(test 一定会考!)

Euler Angles



5

3

Pitch > 0: 相机朝上看 (抬头)

Pitch < 0: 相机朝下看 (低头)

改变 Pitch 会改变:

$$front_y = \sin(pitch)$$

作用:

- 控制纵向视角
- 改变相机所看的方向向量 (Front)

Pitch

Pitch rotates the camera **on the x-axis**. 5. 为什么 Pitch 要限制在 $[-89^\circ, 89^\circ]$?

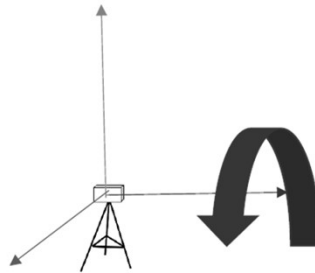
因为:

- Pitch = $+90^\circ$ → 完全向上
- Pitch = -90° → 完全向下
- 再继续会导致相机“翻转”(翻跟头)
→ Gimbal Lock 问题

考试可能会问:

What happens if pitch is not clamped?

答: The camera flips upside-down.



6

Yaw = 围绕 Y 轴旋转 = 左右转头 = 水平旋转

Yaw 决定的是方向向量的水平角度:

- Yaw 增大 → 相机向右转
- Yaw 减小 → 相机向左转

Yaw

◆ Yaw rotates the camera on the **y-axis**.

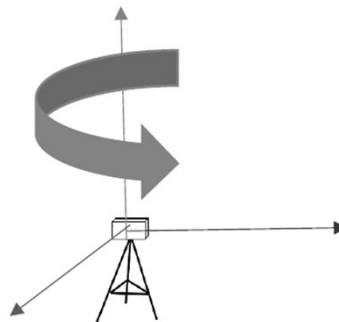
Yaw 影响 Front 向量的 x 和 z 分量:

$$front_x = \cos(yaw) \cos(pitch)$$

$$front_z = \sin(yaw) \cos(pitch)$$

Pitch 控制 y 分量

Yaw 控制 x-z 平面方向



7

Roll = 围绕 Z 轴旋转 = 相机“歪头”

Roll 就像你把头向左倾斜或向右倾斜。

- 围绕 Z 轴
- 改变相机坐标系的“Up”方向
- 但不改变相机所看的“目标方向”（取决于 yaw、pitch）

Roll

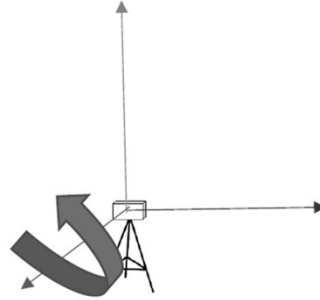
- ◆ Roll rotates the camera on the **z-axis**.

Roll 改变 Up、Right 向量：

- 如果使用 Roll，需要额外绕 Z 轴构建旋转矩阵
- 然后重新计算 Up, Right

但：

★ 在你们的课程和相机代码中，不使用 Roll！
因为 Roll 会让相机倾斜，不好控制。



Model-Viewer Camera

摄像机始终注视着同一个目标，即被观察的模型。我们将模型置于原点 $(0, 0, 0)$ 。

- ◆ The camera constantly looks at the same target which is the model being viewed. We leave the model at the **origin $(0, 0, 0)$** .

✓ 模型固定在世界原点 $(0,0,0)$

你不移动模型。

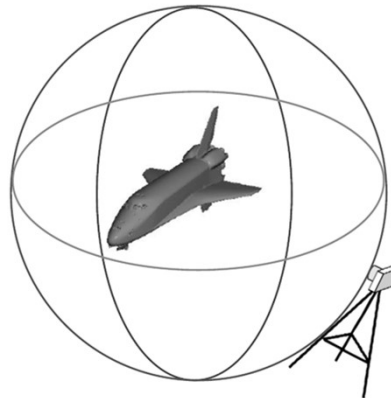
✓ 相机永远“看着”模型的中心

相机的方向始终指向 **target = $(0,0,0)$** 。

✓ 相机围绕模型旋转

你旋转的不是模型，而是相机。

相机在一个球面上（图中的大球）沿着轨迹移动。



这就是 Model-Viewer Camera = Orbit Camera。
9

✓ 相机绕“纬线 (latitude)” 上下移动

纬线就是像地球上的“赤道 + 其他平行圈”。

图示中：

- 模型在原地
- 外面一个球代表“相机运行路径”
- 上下方向的圆弧箭头 → Pitch 改变 (抬头/低头)

Modify Pitch

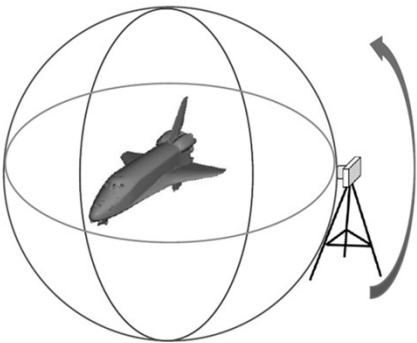
用户可以沿着环绕模型的假想球体的纬度方向上下移动相机。

- ◆ The user can move the camera up and down along the latitude, around an imaginary sphere surrounding the model.

相机在球上沿“上/下方向”运动，形成：

- Pitch 增大：相机绕模型向上移动
- Pitch 减小：相机绕模型向下移动

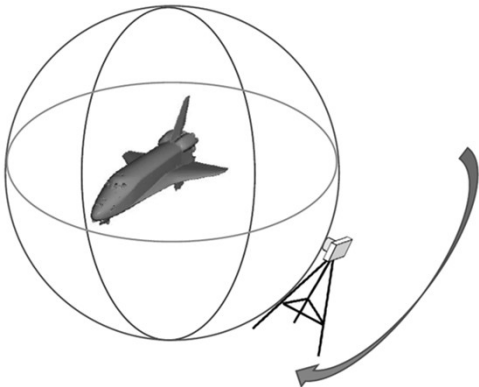
模型不动，相机动。



Modify Yaw

用户沿经线左右移动相机，围绕着一个假想的球体移动，该球体环绕着模型。

- ◆ The user move the camera left and right along the longitude, around an imaginary sphere surrounding the model.



- 模型在球中心
- 相机沿着“左右方向”的大圈旋转
- 左右拉动鼠标 → Yaw 改变 → 相机绕模型水平移动

这就是“轨道相机最经典的旋转方式”。

图里：

- 相机沿着“从模型指向相机”的射线前后移动
- 虚线箭头表示：靠近模型、远离模型

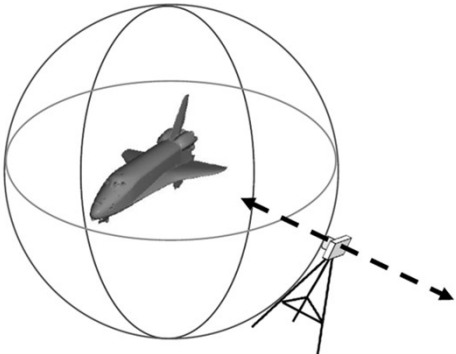
也就是：

- ✓ Radius 增大 → 相机远离模型
- ✓ Radius 减小 → 相机靠近模型

Modify Distance

用户还可以向前或向后调整相机与模型之间的距离，这会改变虚拟球体的半径。

- ◆ The user can also modify the distance of the camera from the model forward and backward, which changes the radius of the imaginary sphere.



The University of Nottingham
UNITED KINGDOM • CHINA • MALAYSIA

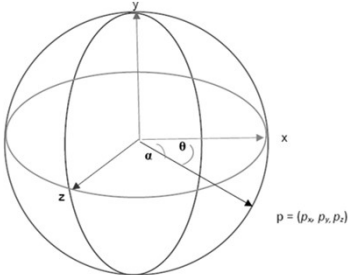
12

12

Implementation of Model-Viewer

- ◆ The Model-Viewer camera is implemented using a unit sphere.

$$p_x = \cos \theta * \cos \alpha$$
$$p_y = \sin \alpha$$
$$p_z = \sin \theta * \cos \alpha$$



🎯 1. 什么是“unit sphere 实现相机”？

意思是：

相机位置被计算为球坐标 (α, θ) → 转换为直角坐标 (px, py, pz)
然后乘以 radius，就得到真实位置。

这是 3D 轨道相机 (Orbit Camera) 的数学基础。

图中：

- α = Pitch (上下角度)
- θ = Yaw (水平角度)

注意！

这里 α 和 θ 是课堂公式里的符号，与前面 slide 的 pitch/yaw 对应：

图中的符号	含义	对应 camera
α	上下角度	pitch
θ	左右角度	yaw

球上的点 p = (px, py, pz) 就是相机的位置 (单位球上)。

13

- θ (theta) = Yaw (水平旋转)
- α (alpha) = Pitch (上下旋转)

用这两个角来计算球面坐标 → 就是相机位置。

模型在原点 (0,0,0)，相机在球面上。

Camera Position

◆ If we use $\theta = \text{Yaw}$ and $\alpha = \text{Pitch}$, then the camera Position is the calculated point on the unit sphere.

The University of Nottingham
UNITED KINGDOM • CHINA • MALAYSIA

14

14

The Front direction of the camera is a vector pointing at the origin.

因为模型在 (0,0,0)，所以：

$$\text{Front} = \text{normalize}(0 - \text{position})$$

$$\text{Front} = \text{normalize}(-\text{position})$$

如果模型位置是 target：

$$\text{Front} = \text{normalize}(\text{target} - \text{position})$$

Camera Front

◆ The Front direction of the camera is a vector pointing at the origin of the space.

The University of Nottingham
UNITED KINGDOM • CHINA • MALAYSIA

15

图里：

- 模型在 (0,0,0)
- 相机在 (px, py, pz)
- Front (F) = 从相机指向模型的箭头

Orbit camera 与 Fly-through 有重要差别：

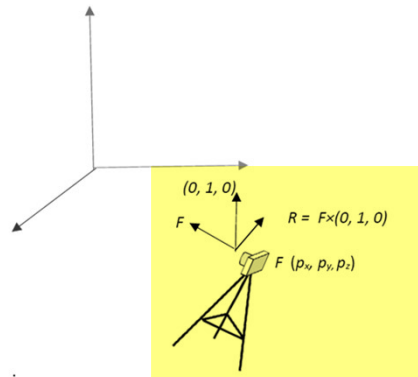
- Fly-through: Front = 根据 yaw/pitch 正向计算出来
- Orbit camera: Front = 根据 “target - position” 反向计算出来

15

Camera Right

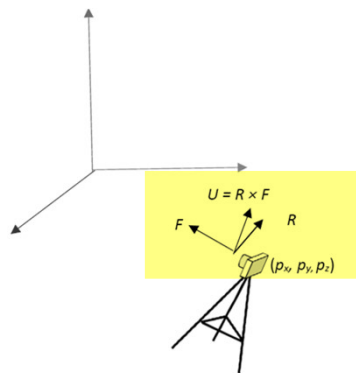
相机的正确方向是相机前向与绝对向上方向 $(0, 1, 0)$ 叉乘所得的向量。

- ◆ The Right direction of the camera is the vector which is the result of cross product of camera Front with absolute up direction $(0, 1, 0)$.



Camera Up

- ◆ The Up direction of the camera is the vector which is the result of cross product of camera Right with camera Front.



因为我们需要一个：

- 与 Right 垂直
- 与 Front 垂直
- 且遵循右手坐标系的 Up 向量

计算顺序必须是：

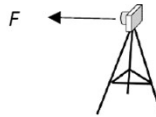
- ✓ Up = Right × Front
- ✗ 非 Front × Right (否则方向反了)

Fly-Through Camera

- ◆ The camera has a Position in world space and a focus direction Front.

相机在世界空间中有一个位置和一个朝向前方的聚焦方向。

Fly-Through：相机在世界里自由走动



改变 Yaw 让相机“左右看”

Yaw = 绕 Y 轴旋转 (转头)

- 鼠标左右移动 → yaw 变化
- yaw 控制 Front 的水平向量

数学公式：

$$front_x = \cos(yaw) \cos(pitch)$$

$$front_z = \sin(yaw) \cos(pitch)$$

就是 FPS 游戏左右看。

Modify Yaw

- ◆ The user can modify the Yaw to alter the focus direction.



Pitch = 上下看 (抬头/低头)

Pitch = 绕 X 轴旋转

Pitch 控制 Front 的垂直方向:

$front_y = \sin(pitch)$

Pitch 必须限制在 $[-89^\circ, 89^\circ]$:

避免相机翻转。

Yaw + Pitch = 完整的视角控制
相机方向 Front = yaw 和 pitch 的组合。

Fly-Through Camera **不使用 Roll**

Modify Pitch

- The user can also modify the Pitch to alter the focus direction.



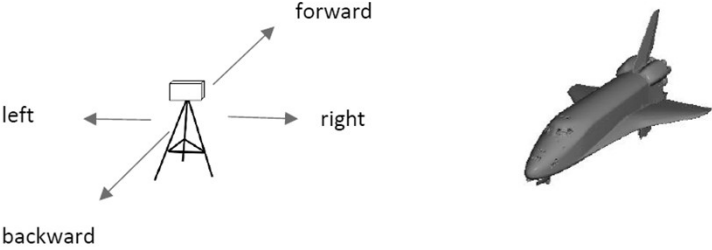


The University of Nottingham
UNITED KINGDOM • CHINA • MALAYSIA

20

Moving Camera

- The user can move the camera forwards, backwards, left and right.



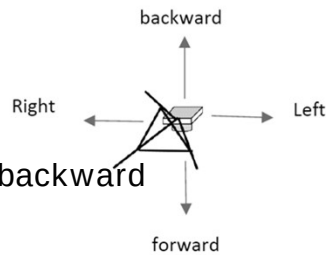


The University of Nottingham
UNITED KINGDOM • CHINA • MALAYSIA

21

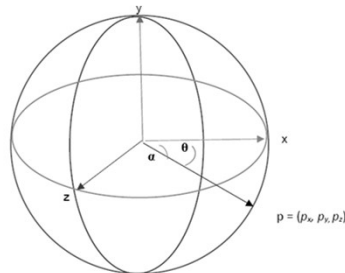
Moving Directions Relative to Front

- ◆ The directions forwards, backwards, left and right are relative to the focus direction Front. 移动方向基于 Front 向量，而不是世界坐标系



Implementation of Fly-Through

- ◆ The Fly-Through camera is also implemented using the unit sphere. 飞越摄像机也是通过单位球体来实现的。
- ◆ We need to imagine that the centre of the unit sphere is at the location of the camera. 我们需要想象单位球体的中心位于摄像机的位置。



2. 为什么 Front 是“球上的点”？

因为：

Fly-Through Camera (FPS Camera)

使用两个角度控制视角：

- **Yaw**: 左右看 = 球面水平角度
- **Pitch**: 上下看 = 球面垂直角度

这两个角唯一地决定球体上的一个点。

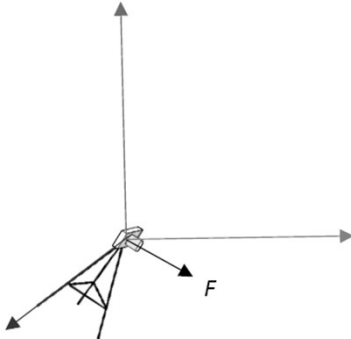
就像：

- 地球上的位置由经纬度决定
- 相机的 Front 由 yaw/pitch 决定

Camera Front

相机的焦点方向是单位球面上的一个点所对应的向量。

- ▶ The focus direction **Front** of the camera is the vector which is the point on the unit sphere.



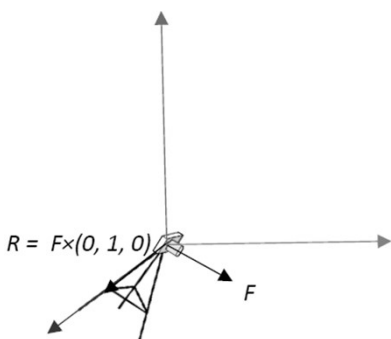
The University of Nottingham
UNITED KINGDOM • CHINA • MALAYSIA

24

24

Camera Right

- ◆ The **Right** direction of the camera is the vector which is the result of **cross product of camera Front with absolute up direction (0, 1, 0)**.



$Right = Front \times WorldUp$

为了保持相机坐标正交：

$$Right = normalize(Front \times (0, 1, 0))$$

- 用世界 Up = (0,1,0)
- 叉乘保证与 Front 垂直

FPS 移动的 right/left 就是用这个向量。

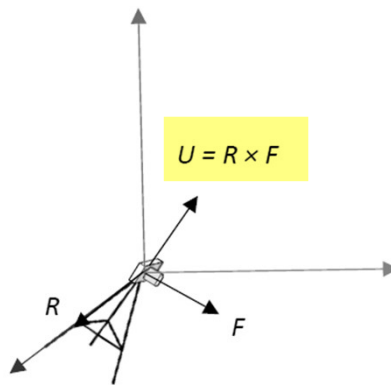
The University of Nottingham
UNITED KINGDOM • CHINA • MALAYSIA

25

25

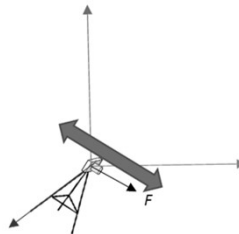
Camera Up

- ◆ The Up direction of the camera is the vector which is the result of cross product of camera Right with camera Front.



Moving Camera Forwards & Backwards

- ◆ The camera can move forwards by
 $Position = Position + Front$
and backwards by
 $Position = Position - Front$



✓ Fly-Through 相机:

- 位置自由移动
- 方向由 yaw/pitch 算出 Front
- $Right = Front \times WorldUp$
- $Up = Right \times Front$
- 前后左右移动都基于 (Front, Right)

✓ 数学核心 (前方向向量 Front):

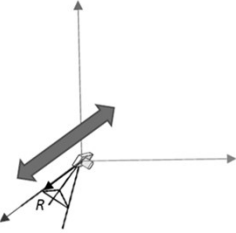
$F_x = \cos(yaw) \cos(pitch)$

$F_y = \sin(pitch)$

$F_z = \sin(yaw) \cos(pitch)$

Moving Camera Right & Left

◆ $Position + Right$
and left by subtracting camera right. $Position = Position - Right$



The University of Nottingham
DOM - CHINA - MALAYSIA

28

✓ 移动公式:

- Position += Front
- Position -= Front
- Position += Right
- Position -= Right

✓ 不使用 Roll

References

COMP3069 Lecture notes: Chapter 8
Cameras

◆ **Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL, Chapter 4**

✓ Orbit 与 Fly-Through 最大区别:

Orbit Camera	Fly-Through Camera
球面 = 位置	球面 = 方向 Front
Position 在球上	Position 是自由的
Front = target - pos	Front = f(yaw,pitch)
不可自由移动	可自由移动